Web (Thymeleaf i SPA)

Тема 7

Як ми можемо додати UI до нашого застосунку?

Способи додати UI до застосунку Spring Boot

- Серверний рендеринг через Thymeleaf формує готовий HTML у контролерах і підходить для адміністративних панелей, форм і каталогів.
- Односторінковий клієнт (React, Angular, Vue) можна зібрати в статику та роздавати тим самим застосунком за патерном BFF зі спільною сесією.
- Окремо розгорнутий клієнтський застосунок звертається до REST API з іншого домену, тому потребує CORS і токенів доступу, як у попередній лекції.
- Гібридний підхід поєднує сторінки Thymeleaf і SPA в одному застосунку, а кожен варіант отримує власний ланцюг безпеки.

Зміст лекції

- Обробка запитів у Spring Web MVC
- Статичні ресурси та HTTP-кешування
- Серверний рендеринг і форми на Thymeleaf
- Безпека веб-інтерфейсу
- Односторінкові клієнти у патерні BFF

Компоненти сервлетного стеку

- Фабрика `ServletWebServerFactory` створює та запускає вбудований сервер Tomcat або Jetty під час старту застосунку.
- Кожен HTTP-запит інкапсулюється в об'єкти `HttpServletRequest` та `HttpServletResponse`.
- Ланцюг фільтрів, зокрема `SecurityFilterChain` з попередньої лекції, виконує автентифікацію та перевірки безпеки до виклику `DispatcherServlet`.
- Сервлет `DispatcherServlet` є єдиною точкою входу (патерн Front Controller) і через `HandlerMapping` знаходить обробник запиту.
- Контролер повертає ім'я представлення для рендерингу, а запити до файлів обробляє `ResourceHttpRequestHandler`.

Документація: Spring Web MVC DispatcherServlet

Модель потоків сервлет-контейнера

- Модель Thread-per-Request закріплює за кожним запитом окремий потік на весь час обробки.
- Tomcat за замовчуванням обслуговує до 200 запитів одночасно (`server.tomcat.threads.max`), а нові з'єднання понад ліміт очікують у черзі ОС (`server.tomcat.accept-count`).
- Блокувальний виклик бази даних чи зовнішнього сервісу утримує потік, тому повільна залежність швидко вичерпує пул.
- У Java 25 властивість `spring.threads.virtual.enabled=true` переводить обробку запитів на віртуальні потоки, які на час очікування I/O звільняють потік операційної системи для інших запитів.

Схема обробки запиту у Spring Web MVC



Роздача статики через ResourceHttpRequestHandler

- Spring Boot роздає файли з `classpath:/static/`, `/public/`, `/resources/` та `/META-INF/resources/` (WebJars) від кореня URL.
- Обробник `ResourceHttpRequestHandler` визначає MIME-тип файлу та підтримує запити діапазонів байтів (Byte-Range).
- Обробник відхиляє шляхи з `../` та їхні закодовані варіанти, захищаючи від обходу каталогу (Directory Traversal).
- Статичні каталоги призначені для публічної роздачі, тому конфіденційні файли в них не розміщуються.

HTTP-кешування та заголовок Cache-Control

- Клієнтське кешування усуває повторне передавання незмінних CSS та JS файлів мережею.
- Директива `max-age=<секунди>` задає термін придатності без звернення до сервера, а `public` дозволяє зберігання копії в CDN.
- Директива `no-cache` дозволяє зберігати копію, але перед використанням браузер питає сервер, чи вона ще актуальна.
- Директива `no-store` забороняє зберігання відповіді, і Spring Security за замовчуванням додає її до відповідей без явних налаштувань кешування.

Специфікація: RFC 9111 HTTP Caching

Приклад конфігурації кешування статичних ресурсів

```
@Configuration(proxyBeanMethods = false)
public class WebMvcAssetConfiguration implements WebMvcConfigurer {

    @Override
    public void addResourceHandlers(ResourceHandlerRegistry registry) {
        registry.addResourceHandler("/assets/**")
                .addResourceLocations("classpath:/static/assets/")
                .setCacheControl(CacheControl.maxAge(365, TimeUnit.DAYS).cachePublic());
    }
}
```

Пояснення налаштування обробника ресурсів

- Метод `addResourceHandler("/assets/**")` задає URL-шаблон для перехоплення запитів.
- Метод `addResourceLocations` прив'язує URL до фізичного каталогу всередині classpath.
- Клас `CacheControl` формує заголовок `Cache-Control: max-age=31536000, public`, тобто кешування на 1 рік.
- Річний термін кешування безпечний лише для файлів із хешем вмісту в імені, інакше клієнти не отримають оновлень.

Умовна валідація ETag та ShallowEtagHeaderFilter

- Заголовок `ETag` містить ідентифікатор версії ресурсу, який клієнт повертає в заголовку `If-None-Match`.
- При збігу сервер відповідає статусом HTTP 304 Not Modified без тіла відповіді.
- Фільтр `ShallowEtagHeaderFilter` обчислює ETag як хеш MD5 уже згенерованої відповіді, тому заощаджує лише трафік, а не CPU на рендеринг.
- Сторінки з CSRF-токеном у формі отримують новий ETag на кожен запит, тому для них фільтр неефективний.

Ланцюг ресурсів із попередньо стисненими файлами Brotli

```
@Configuration(proxyBeanMethods = false)
public class WebMvcResourceChainConfig implements WebMvcConfigurer {

    @Override
    public void addResourceHandlers(ResourceHandlerRegistry registry) {
        registry.addResourceHandler("/css/**")
                .addResourceLocations("classpath:/static/css/")
                .setCacheControl(CacheControl.maxAge(365, TimeUnit.DAYS).cachePublic())
                .resourceChain(true)
                .addResolver(new EncodedResourceResolver())
                .addResolver(new VersionResourceResolver().addContentVersionStrategy("/**"));
    }
}
```

Пояснення роботи ланцюга обробників ресурсів

- Попередньо стиснені файли `.br` та `.gz` створюються заздалегідь утилітою стиснення під час збірки (`gzip` , `brotli`) та зменшують розмір текстових ресурсів на 70–80%.
- Обробник `EncodedResourceResolver` віддає файл `.br` чи `.gz` відповідно до заголовка `Accept-Encoding` без витрат CPU на стиснення під час запиту.
- Обробник `VersionResourceResolver` додає хеш вмісту до імені файлу: браузер може зберігати файл весь рік із `maxAge(365, DAYS)` , а після зміни вмісту отримає нове ім'я і завантажить нову версію.
- Виклик `resourceChain(true)` запам'ятовує, який файл відповідає якому URL, щоб не шукати його повторно.

Версіонування ресурсів за хешем вмісту

- Підхід Cache Busting змінює URL файлу при зміні його вмісту, тому браузер завантажує нову версію попри річний кеш.
- Параметри запиту на кшталт `?v=2` ненадійні, оскільки частина проксі та CDN їх ігнорує.
- Spring обчислює хеш MD5 вмісту та вбудовує його в ім'я файлу, перетворюючи `style.css` на `style-f98e421d57902a256ad04856129445ca.css`.
- Незмінний файл зберігає ім'я та клієнтський кеш, а будь-яка правка автоматично змінює хеш.

Приклад конфігурації версіонування ресурсів

```
# Spring Boot application.properties
spring.web.resources.chain.enabled=true
spring.web.resources.chain.strategy.content.enabled=true
spring.web.resources.chain.strategy.content.paths=/**
spring.web.resources.chain.cache=true
```

Пояснення механізму автоматичного версіонування

- Параметр `chain.enabled` вмикає обробку статики ланцюгом обробників (стиснення, версіонування), а `strategy.content` додає версіонування за хешем для шляхів `/**`.
- Параметр `chain.cache=true` зберігає в пам'яті відповідність оригінальних та хешованих імен файлів.
- Компонент `ResourceUrlProvider` разом із фільтром `ResourceUrlEncodingFilter` переписує посилання, сформовані через `@{...}` у Thymeleaf.
- Запис `<link th:href="@{/css/app.css}" rel="stylesheet"/>` автоматично отримує хешоване ім'я файлу без ручного контролю версій.
- Ті самі налаштування можна задати в коді через `VersionResourceResolver`, як у прикладі з Brotli.

Статичні ресурси та користувацькі файли

- Статика застосунку (CSS, JavaScript, логотипи) пакується в JAR під час збірки і не змінюється до наступного випуску.
- Файли, які завантажують користувачі (фото, документи, вкладення), з'являються під час роботи, тому записати їх у JAR неможливо.
- Такі файли зберігають у каталозі на диску сервера поза JAR або в об'єктному сховищі (S3, MinIO).
- Каталог, підключений як статика через префікс `file:`, доступний усім без входу, тому користувацькі файли віддає контролер із перевіркою прав.

Роздача користувацьких файлів поза JAR

```
@GetMapping("/cameras/{id}/snapshot")
public ResponseEntity<Resource> snapshot(@PathVariable String id) {
    return snapshotService.loadSnapshot(id)
            .map(file -> ResponseEntity.ok()
                    .contentType(MediaTypeFactory.getMediaType(file).orElseThrow())
                    .cacheControl(CacheControl.noStore())
                    .body(file))
            .orElse(ResponseEntity.notFound().build());
}

private Path resolve(String fileName) {
    Path file = baseDir.resolve(fileName).normalize();
    if (!file.startsWith(baseDir)) {
        throw new SnapshotRejectedException("Некоректний шлях до файлу");
    }
    return file;
}
```

Пояснення роздачі користувацьких файлів

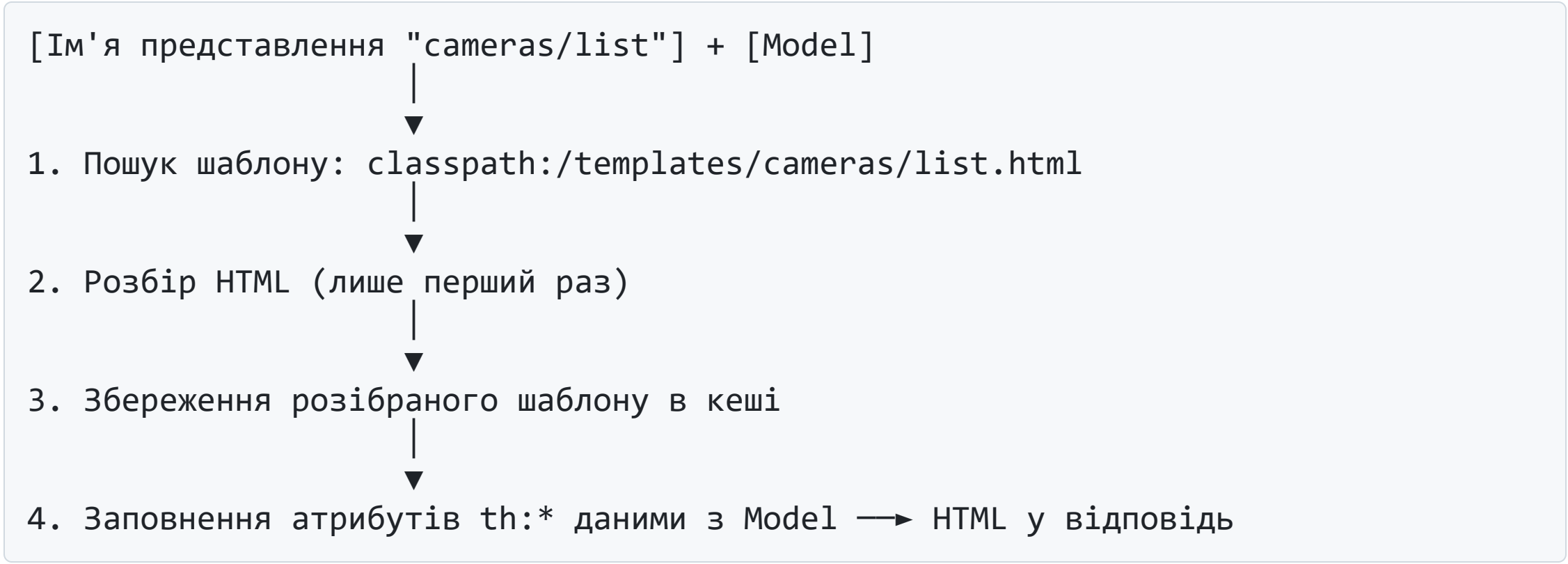
- Файли зберігаються в каталозі поза JAR, заданому властивістю, тому не зникають після оновлення або повторної збірки застосунку.
- Підключення каталогу через `addResourceLocations("file:...")` роздало б файли всім без перевірки прав, тому файл віддає контролер.
- Методи `normalize()` і `startsWith(baseDir)` не дають вийти за межі каталогу через `../` в імені файлу.
- Тип завантаженого файлу визначається за його сигнатурою, а не за заголовком `Content-Type`, який задає клієнт.
- Заголовок `Cache-Control: no-store` забороняє браузеру та проксі-серверам зберігати копії приватних файлів.

Концепція природних шаблонів у Thymeleaf

- Шаблони Thymeleaf є валідними документами стандарту HTML5 і відкриваються у браузері без сервера.
- Динамічна поведінка задається власними атрибутами простору імен `th:*`.
- Статичні прототипні дані всередині тегів замінюються обчисленими значеннями під час запиту.
- Дизайнери та розробники працюють з одними й тими самими файлами розмітки.
- На відміну від застарілого JSP, шаблон не компілюється в сервлет і працює у виконуваному JAR-архіві Spring Boot.

Документація: Thymeleaf Official Documentation

Як Thymeleaf рендерить сторінку



Критична роль кешування розібраних шаблонів

- Розбір HTML — найдорожчий крок рендерингу, тому без кешу він повторювався б на кожен запит і перевантажував CPU.
- Налаштування `spring.thymeleaf.cache=true` є обов'язковим для промислового середовища.
- Вимкнення кешування дозволяється виключно в режимі локальної розробки для гарячого перезавантаження.

Веб-інтерфейс у модульному моноліті Spring Modulith

```
camera                  <-- модуль: публічний сервіс, DTO, події
  ├── internal          <-- реалізація та сховище
  ├── web               <-- контролери Thymeleaf і форми
  └── api               <-- REST-контролери для SPA та зовнішніх клієнтів
```

- Контролери та форми — вхідні точки модуля з боку вебу (адаптери): вони лише перетворюють HTTP-запит на виклик сервісу та розміщуються у вкладеному пакеті модуля.
- Адаптер звертається лише до публічного сервісу модуля, а `ApplicationModules.verify()` забороняє іншим модулям використовувати класи адаптера.
- Спільні елементи інтерфейсу (сторінка входу, макети, роздача SPA) виносяться в окремий модуль конфігурації без бізнес-логіки.

Форма введення з валідацією JSR-380 для камери

```
import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.Pattern;
import jakarta.validation.constraints.Size;

public record CreateCameraForm(
        @NotBlank(message = "Ідентифікатор камери є обов’язковим")
        @Size(min = 3, max = 20, message = "Ідентифікатор має містити від 3 до 20 символів")
        String cameraId,

        @NotBlank(message = "Локація є обов’язковою")
        String location,

        @Pattern(regexp = "^([0-9]{1,3}\\.){3}[0-9]{1,3}$", message = "Некоректна IPv4 адреса")
        String ipAddress
) {}
```

Контролер представлення CameraWebController

```
@Controller
@RequestMapping("/cameras")
public class CameraWebController {

    private final CameraService cameraService;

    public CameraWebController(CameraService cameraService) {
        this.cameraService = cameraService;
    }

    @GetMapping
    public String listCameras(Model model) {
        model.addAttribute("cameras", cameraService.getAllCameras());
        return "cameras/list";
    }
```

Реалізація обробки форми за патерном Post-Redirect-Get

```
    @GetMapping("/new")
    public String showCreateForm(Model model) {
        model.addAttribute("form", new CreateCameraForm("", "", ""));
        return "cameras/create-form";
    }

    @PostMapping
    public String handleCreateCamera(
            @Valid @ModelAttribute("form") CreateCameraForm form,
            BindingResult bindingResult,
            RedirectAttributes redirectAttributes) {

        if (bindingResult.hasErrors()) {
            return "cameras/create-form";
        }

        cameraService.registerCamera(form.cameraId(), form.location(), form.ipAddress());
        redirectAttributes.addFlashAttribute("successMessage", "Камеру успішно підключено.");
        return "redirect:/cameras";
    }
}
```

Пояснення роботи контролера та патерну PRG

- Контролер отримує лише публічний `CameraService` і не знає про сховище та реалізацію модуля.
- Метод GET `/new` передає в модель порожню форму, а `th:object` прив'язує до неї поля шаблону.
- За помилок валідації `BindingResult` повертає ту саму сторінку з повідомленнями під полями без перенаправлення.
- Патерн PRG захищає від дублювання реєстрації при оновленні сторінки користувачем.

Шаблон списку камер у Thymeleaf (list.html)

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org" lang="uk">
<head>
    <meta charset="UTF-8">
    <title>CCTV Моніторинг</title>
    <link rel="stylesheet" th:href="@{/css/bootstrap.min.css}"/>
</head>
<body class="container py-4">
    <div th:if="${successMessage}" class="alert alert-success" th:text="${successMessage}"></div>
    <h2>Підключені камери</h2>
    <a th:href="@{/cameras/new}" class="btn btn-primary mb-3">Додати камеру</a>
```

Таблиця камер з чергуванням рядків та статусами

```
    <table class="table table-bordered">
        <thead>
            <tr><th>#</th><th>ID</th><th>Локація</th><th>IP</th><th>Статус</th></tr>
        </thead>
        <tbody>
            <tr th:each="cam, stat : ${cameras}" th:class="${stat.odd} ? 'table-light' : ''">
                <td th:text="${stat.count}">1</td>
                <td th:text="${cam.cameraId}">CAM-01</td>
                <td th:text="${cam.location}">Хол</td>
                <td th:text="${cam.ipAddress}">192.168.1.10</td>
                <td th:switch="${cam.active}">
                    <span th:case="true" class="badge bg-success">Активна</span>
                    <span th:case="false" class="badge bg-danger">Вимкнена</span>
                </td>
            </tr>
        </tbody>
    </table>
</body>
</html>
```

Шаблон форми реєстрації камери (create-form.html)

```
<form th:action="@{/cameras}" th:object="${form}" method="post">
    <div class="mb-3">
        <label for="cameraId" class="form-label">ID камери</label>
        <input type="text" id="cameraId" th:field="*{cameraId}" class="form-control"/>
        <div class="text-danger small" th:if="${#fields.hasErrors('cameraId')}"
             th:errors="*{cameraId}">Помилка ID</div>
    </div>
    <div class="mb-3">
        <label for="location" class="form-label">Локація</label>
        <input type="text" id="location" th:field="*{location}" class="form-control"/>
        <div class="text-danger small" th:if="${#fields.hasErrors('location')}"
             th:errors="*{location}">Помилка локації</div>
    </div>
```

Поля адреси та кнопки форми реєстрації

```
    <div class="mb-3">
        <label for="ipAddress" class="form-label">IP-адреса</label>
        <input type="text" id="ipAddress" th:field="*{ipAddress}" class="form-control"/>
        <div class="text-danger small" th:if="${#fields.hasErrors('ipAddress')}"
             th:errors="*{ipAddress}">Помилка IP</div>
    </div>
    <button type="submit" class="btn btn-success">Зареєструвати</button>
    <a th:href="@{/cameras}" class="btn btn-secondary">Скасувати</a>
</form>
```

Багатомовність інтерфейсу

```
# messages.properties (українська за замовчуванням)
cameras.heading=Підключені камери
camera.location.required=Локація є обов’язковою
# messages_en.properties
cameras.heading=Connected cameras
camera.location.required=Location is required
```

```
<h1 th:text="#{cameras.heading}">Підключені камери</h1>
<a href="?lang=en">EN</a>
```

- Тексти виносяться у `messages*.properties` : шаблон бере їх виразом `#{...}` , а валідація — ключем `{camera.location.required}` .
- `CookieLocaleResolver` запам'ятовує мову в куці, а `LocaleChangeInterceptor` перемикає її параметром `?lang=en` .

Перехід від захисту REST API до захисту веб-інтерфейсу

- Попередня лекція захищала REST API за допомогою stateless JWT у заголовку `Authorization` з вимкненим CSRF.
- Браузерний інтерфейс на Thymeleaf зберігає автентифікацію в сесії, а браузер додає сесійну куку до кожного запиту автоматично.
- Автоматична передача куки робить сторінки вразливими до CSRF, тому для веб-інтерфейсу захист CSRF залишається увімкненим.
- Далі ми розглянемо окремий ланцюг безпеки для UI, форму входу, CSRF у формах, діалект `sec:`, контексти екранування, SSTI та заголовки безпеки.

Ланцюг безпеки REST API для зовнішніх клієнтів

```java
@Configuration(proxyBeanMethods = false)
@EnableWebSecurity
@EnableMethodSecurity
public class SecurityConfiguration {

    @Bean
    @Order(1)
    SecurityFilterChain apiFilterChain(HttpSecurity http, JwtAuthenticationFilter jwtFilter) throws Exception {
        return http
                .securityMatcher("/api/**")
                .csrf(AbstractHttpConfigurer::disable)
                .sessionManagement(session -> session.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
                .authorizeHttpRequests(auth -> auth.anyRequest().authenticated())
                .addFilterBefore(jwtFilter, UsernamePasswordAuthenticationFilter.class)
                .build();
    }
```

Ланцюг безпеки браузерного інтерфейсу з формою входу

```
    @Bean
    @Order(3)
    SecurityFilterChain webFilterChain(HttpSecurity http) throws Exception {
        return http
                .authorizeHttpRequests(auth -> auth
                        .requestMatchers("/login", "/css/**", "/js/**", "/error").permitAll()
                        .requestMatchers(HttpMethod.POST, "/cameras").hasRole("ADMIN")
                        .anyRequest().authenticated())
                .formLogin(form -> form
                        .loginPage("/login")
                        .defaultSuccessUrl("/cameras"))
                .logout(logout -> logout.logoutSuccessUrl("/login?logout"))
                .build();
    }
}
```

Пояснення конфігурації двох ланцюгів безпеки

- Ланцюг з `@Order(1)` та `securityMatcher("/api/**")` обробляє лише REST API і залишається stateless з JWT, як у попередній лекції.
- Ланцюг з `@Order(3)` без `securityMatcher` обробляє решту запитів, зберігає автентифікацію в `HttpSession` і не вимикає CSRF, а місце `@Order(2)` займе ланцюг односторінкового клієнта.
- Метод `formLogin` з `loginPage("/login")` замінює стандартну сторінку входу власним шаблоном Thymeleaf.
- Вихід із системи виконується лише запитом POST із токеном CSRF, оскільки посилання GET на `/logout` можна підробити зі стороннього сайту.
- Правило `hasRole("ADMIN")` для POST `/cameras` доповнюється анотацією `@PreAuthorize` на рівні сервісу, як у попередній лекції.

Шаблон сторінки входу login.html

```
<form th:action="@{/login}" method="post">
    <div th:if="${param.error}" class="alert alert-danger">Невірне ім'я користувача або пароль.</div>
    <div th:if="${param.logout}" class="alert alert-info">Ви вийшли із системи.</div>
    <input type="text" name="username" autocomplete="username" required/>
    <input type="password" name="password" autocomplete="current-password" required/>
    <button type="submit">Увійти</button>
</form>
```

- Сторінка реєструється без окремого контролера викликом `registry.addViewController("/login").setViewName("login")`.
- Поля `username` та `password` відповідають параметрам, які очікує фільтр `UsernamePasswordAuthenticationFilter`.
- Однакове повідомлення для невідомого користувача та неправильного пароля не розкриває наявність облікових записів.

Захист форм від CSRF через th:action

- Атака CSRF змушує браузер автентифікованого користувача надіслати запит зі стороннього сайту разом із сесійною кукою.
- Spring Security вимагає токен CSRF для запитів POST, PUT, PATCH та DELETE і відхиляє запит без токена статусом 403.
- Атрибут `th:action` автоматично додає до форми приховане поле `_csrf` з поточним токеном.
- Звичайний атрибут `action` без префікса `th:` токен не додає, тому така форма отримує відповідь 403.
- Дані змінюються лише через POST, бо запити GET Spring Security на CSRF не перевіряє.

Згенерована форма з прихованим токеном CSRF

```
<!-- Шаблон -->
<form th:action="@{/cameras}" th:object="${form}" method="post">

<!-- Результат рендерингу -->
<form action="/cameras" method="post">
    <input type="hidden" name="_csrf" value="hk3Lx7...Qz0"/>
```

- Значення токена в HTML змінюється з кожною відповіддю, тому його не можна підібрати, аналізуючи трафік.
- Для запитів JavaScript той самий токен доступний у шаблоні через `${_csrf.token}` та `${_csrf.headerName}`.

Діалект sec для відображення залежно від ролі

```
<html xmlns:th="http://www.thymeleaf.org"
      xmlns:sec="http://www.thymeleaf.org/extras/spring-security" lang="uk">

<span sec:authentication="name">operator</span>

<a sec:authorize="hasRole('ADMIN')" th:href="@{/cameras/new}">Додати камеру</a>

<form th:action="@{/logout}" method="post">
    <button type="submit">Вийти</button>
</form>
```

Пояснення діалекту sec та його обмежень

- Діалект підключається залежністю `org.thymeleaf.extras:thymeleaf-extras-springsecurity6`, яку Spring Boot автоматично реєструє в шаблонізаторі.
- Атрибут `sec:authorize` приймає ті самі вирази, що й `@PreAuthorize`, зокрема `hasRole`, `hasAuthority` та `isAuthenticated()`.
- Атрибут `sec:authentication="name"` виводить ім'я поточного користувача з екрануванням, як `th:text`.
- Приховування кнопки не є захистом, тому серверне правило `hasRole("ADMIN")` та `@PreAuthorize` залишаються обов'язковими.
- Кнопка виходу оформлюється формою POST, щоб `th:action` додав до неї токен CSRF.

Загроза міжсайтового скриптингу у динамічному UI

- Атака Cross-Site Scripting (XSS) полягає у впровадженні шкідливого скрипту в розмітку сайту.
- Браузер виконує переданий тег `<script>` у довіреному контексті сесії постраждалого користувача.
- Наслідками атаки є перехоплення сесійних ідентифікаторів, підробка дій або модифікація DOM.
- Захист вимагає обов'язкового екранування спецсимволів HTML (`<`, `>`, `&`, `"`, `'`).

Документація: OWASP Cross Site Scripting (XSS) Prevention

Екранування символів директивами th:text та th:utext

- Директива `th:text` автоматично конвертує небезпечні символи у безпечні сутності HTML (`<` , `>`).
- Директива `th:utext` вставляє текст без жодної фільтрації та становить пряму загрозу XSS.
- Використання `th:utext` суворо заборонено для будь-яких користувацьких полів.
- Будь-який форматований ввід користувача вимагає санітизації бібліотекою на кшталт OWASP Java HTML Sanitizer.

Контексти екранування у шаблонах Thymeleaf

- Вираз `[[${...}]]` у блоці `th:inline="javascript"` екранує значення як рядковий літерал JavaScript, а `[(${...})]` вставляє його без екранування.
- Атрибут `th:href="${url}"` екранує лапки, але не перевіряє схему, тому значення `javascript:alert(1)` з користувацьких даних виконає скрипт.
- В атрибутах подій (`th:onclick` тощо) Thymeleaf дозволяє лише числа та логічні значення, тому рядок користувача туди не потрапить.
- Посилання з користувацьких даних перевіряються на сервері за переліком дозволених схем `http` та `https`.

Мова виразів SpEL та обмежений режим Thymeleaf 3.1

- Вирази `${...}` у Spring-інтеграції Thymeleaf обчислюються рушієм SpEL із доступом до бінів та моделі.
- Впровадження користувацьких даних у текст виразу створює загрозу SpEL Injection, тобто виконання довільного коду на сервері.
- Обмежений режим Thymeleaf 3.1 забороняє у виразах виклики методів, конструкторів і `T(...)` для системних класів Java (`java.*`, `jakarta.*`, `jdk.*`).
- Об'єкти `#request`, `#response`, `#session` і `#servletContext` вилучено з виразів, тому потрібні дані передаються через модель.
- Обмежений режим мав відомі обходи у версіях 3.1.x і не захищає від підстановки параметрів запиту в ім'я шаблону, тому SSTI залишається реальною загрозою.

Впровадження шаблонів через ім'я представлення

```
// Небезпечно: ім'я шаблону формується з параметра запиту
@GetMapping("/cameras/view")
public String view(@RequestParam String layout) {
    return "cameras/" + layout;
}

// Безпечно: дозволені значення перелічені явно
@GetMapping("/cameras/view")
public String view(@RequestParam CameraView view) {
    return switch (view) {
        case CARDS -> "cameras/cards";
        case TABLE -> "cameras/list";
    };
}
```

Пояснення загрози SSTI та відкритого перенаправлення

- Якщо рядок, який повертає контролер, містить `__${...}__`, Thymeleaf обчислює цей вираз ще до пошуку шаблону.
- Підставлений у параметр вираз SpEL призводить до віддаленого виконання коду на сервері (SSTI).
- Thymeleaf намагається блокувати такі вирази, але цю перевірку вже обходили, тому надійний захист лише один: перелік дозволених значень (`enum`).
- Перелік `CameraView` обмежує вхідні значення, а невідоме значення Spring відхиляє помилкою перетворення типу.
- Конкатенація `"redirect:" + url` з параметра запиту так само небезпечна, оскільки відкриває перенаправлення на сторонній сайт (Open Redirect).

Заголовки безпеки HTTP для веб-інтерфейсу

```
.headers(headers -> headers
        .contentSecurityPolicy(csp -> csp.policyDirectives(
                "default-src 'self'; img-src 'self' data:; "
                + "script-src 'self'; style-src 'self'; frame-ancestors 'none'"))
        .referrerPolicy(referrer -> referrer.policy(
                ReferrerPolicyHeaderWriter.ReferrerPolicy.STRICT_ORIGIN_WHEN_CROSS_ORIGIN)))
```

Пояснення заголовків безпеки

- Spring Security за замовчуванням додає `X-Content-Type-Options: nosniff`, `X-Frame-Options: DENY`, заборону кешування сторінок та HSTS (браузер звертається до сайту лише через HTTPS).
- Політика CSP забороняє вбудовані скрипти та ресурси зі сторонніх доменів, тому впроваджений через XSS тег `<script>` не виконається.
- Директива `frame-ancestors 'none'` забороняє вбудовування сторінки у фрейми та захищає від атаки clickjacking.
- Значення `'unsafe-inline'` знову дозволяє вбудовані скрипти та стилі й фактично вимикає захист CSP, тому його не використовують.
- Строга CSP несумісна з вбудованими `<style>` та бібліотеками з CDN, тому Bootstrap підключається з локальних статичних ресурсів, а директива `img-src 'self' data:` дозволяє його вбудовані SVG-іконки.

Сторінки помилок та витік технічної інформації

- Стандартна сторінка Whitelabel з трасуванням стеку розкриває версії бібліотек та внутрішню структуру застосунку.
- Властивості `server.error.include-stacktrace` та `server.error.include-message` мають за замовчуванням значення `never` і не вмикаються в промисловому середовищі.
- Шаблони `templates/error/403.html`, `404.html` та `5xx.html` автоматично відображаються для відповідних статусів.
- Шлях `/error` відкривається в ланцюзі безпеки, інакше помилка анонімного користувача перетворюється на перенаправлення на сторінку входу.

Тестування захисту форм у MockMvc

```
@SpringBootTest
@AutoConfigureMockMvc
class CameraWebSecurityTest {

    @Autowired
    MockMvc mvc;

    @Test
    @WithMockUser(roles = "ADMIN")
    void postWithoutCsrfIsForbidden() throws Exception {
        mvc.perform(post("/cameras").param("cameraId", "CAM-01"))
           .andExpect(status().isForbidden());
    }

    @Test
    @WithMockUser(roles = "ADMIN")
    void postWithCsrfRedirectsToList() throws Exception {
        mvc.perform(post("/cameras").with(csrf())
                .param("cameraId", "CAM-01").param("location", "Хол").param("ipAddress", "10.0.0.1"))
           .andExpect(redirectedUrl("/cameras"));
    }
}
```

Пояснення тестів безпеки форм

- Запит POST без токена отримує 403 навіть від адміністратора, що підтверджує роботу захисту CSRF.
- Стартер `spring-boot-starter-security-test` підключає Spring Security до MockMvc, тому `@WithMockUser` та `with(csrf())` працюють без ручного налаштування.
- Анотація `@WithMockUser(roles = "ADMIN")` підставляє автентифікованого користувача без проходження форми входу.
- Аналогічно перевіряється, що анонімний запит GET `/cameras` перенаправляє на сторінку `/login`.

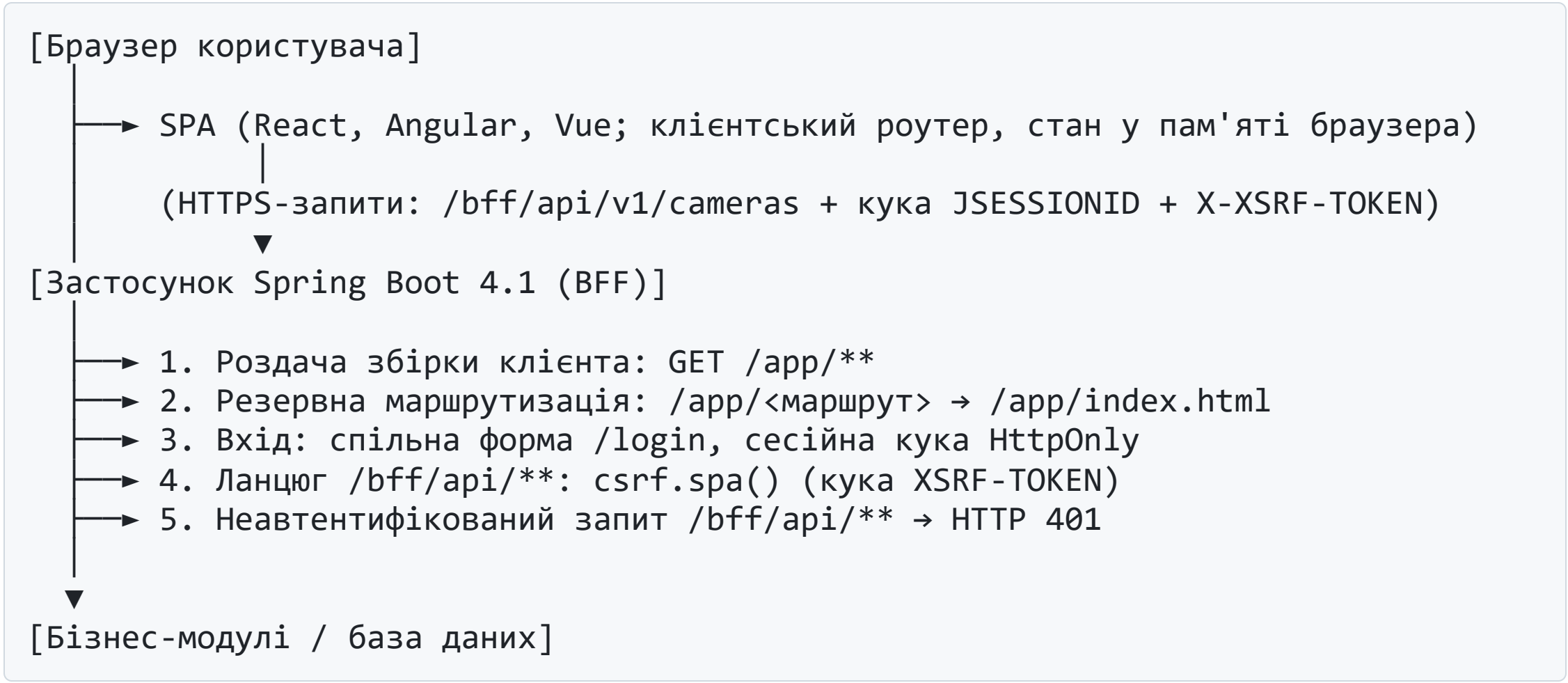
Перехід до архітектури SPA на базі Spring

- Складні інтерактивні клієнти (панелі моніторингу в реальному часі, редактори, інтерактивні карти) оновлюють частини сторінки без перезавантаження.
- Односторінкові додатки (SPA) на React, Angular, Vue чи Svelte переносять управління DOM безпосередньо у браузер.
- Далі ми розглянемо односторінковий клієнт у тому самому застосунку: він отримує власний префікс, власний ланцюг безпеки та спільну з Thymeleaf сесію.

Архітектурний патерн Backend For Frontend (BFF)

- Сервер Spring Boot виступає шлюзом між односторінковим клієнтом та бізнес-модулями застосунку.
- Сервер керує користувацькими сесіями через захищені браузерні куки без передачі токенів у JavaScript.
- Сервер надає клієнту окремий API `/bff/api` лише з потрібними йому даними.

Схема взаємодії SPA та Spring Boot BFF



Загрози безпеки SPA та вразливість LocalStorage

- Збереження JWT токенів доступу у сховищі `localStorage` є небезпечним.
- Вміст `localStorage` читає будь-який скрипт на сторінці, тому перша ж XSS-вразливість чи скомпрометована стороння бібліотека призводить до крадіжки токена.
- Патерн BFF вирішує проблему, залишаючи автентифікацію на сервері та використовуючи сесійну куку `HttpOnly`.
- Кука `HttpOnly` захищає від викрадення сесії, але XSS однаково дозволяє виконувати запити від імені користувача, тому CSP та екранування залишаються обов'язковими.

Роздача збірки SPA із каталогу classpath

- Клієнтський проєкт збирається інструментом свого фреймворку (Vite, Angular CLI, Webpack) у набір статичних файлів.
- Плагін збірки (`frontend-maven-plugin` або завдання Gradle) копіює зібрані файли до каталогу `static/app/` у classpath.
- Клієнт збирається з базовою адресою `/app/`, тому всі його ресурси та маршрути мають спільний префікс.
- Такий підхід дозволяє розгортати повноцінний веб-застосунок у вигляді єдиного самодостатнього JAR-файлу.

Проблема клієнтської маршрутизації SPA

- Клієнтські роутери (React Router, Angular Router, Vue Router) використовують HTML5 History API для перемикання екранів без оновлення.
- Прямий перехід користувача за адресою `/app/cameras/new` ініціює запит GET до сервера Spring.
- Оскільки на сервері відсутній такий статичний файл або контролер, Spring повертає помилку HTTP 404.
- Сервер зобов'язаний віддавати `index.html` для невідомих маршрутів під префіксом клієнта.

Резервна маршрутизація SPA через PathResourceResolver

```
@Configuration(proxyBeanMethods = false)
public class SpaWebRoutingConfiguration implements WebMvcConfigurer {

    @Override
    public void addResourceHandlers(ResourceHandlerRegistry registry) {
        registry.addResourceHandler("/app/**")
                .addResourceLocations("classpath:/static/app/")
                .resourceChain(true)
                .addResolver(new PathResourceResolver() {
                    @Override
                    protected Resource getResource(String resourcePath, Resource location) throws IOException {
                        Resource requested = super.getResource(resourcePath, location);
                        if (requested != null) {
                            return requested;
                        }
                        return resourcePath.contains(".") ? null : new ClassPathResource("static/app/index.html");
                    }
                });
    }
}
```

Пояснення конфігурації маршрутизації SPA

- Обробник ресурсів для `/app/**` обслуговує лише односторінковий клієнт і не перетинається зі сторінками Thymeleaf та REST-контролерами.
- Наявний файл повертається викликом `super.getResource`, який зберігає перевірку на обхід каталогу.
- Шлях без розширення отримує `index.html`, а відсутній файл із розширенням повертає 404.
- Корінь `/app/` не доходить до `PathResourceResolver`, тому потребує окремого рядка `registry.addViewController("/app/").setViewName("forward:/app/index.html")`.
- Шаблон `/**/{segment}` не використовується, оскільки `PathPatternParser` дозволяє `**` лише в кінці шаблону.

Ланцюг безпеки BFF для односторінкового клієнта

```
@Bean
@Order(2)
SecurityFilterChain bffFilterChain(HttpSecurity http) throws Exception {
    return http
            .securityMatcher("/bff/api/**")
            .csrf(csrf -> csrf.spa())
            .authorizeHttpRequests(auth -> auth
                    .requestMatchers(HttpMethod.POST, "/bff/api/v1/cameras").hasRole("ADMIN")
                    .anyRequest().authenticated())
            .exceptionHandling(exceptions -> exceptions
                    .authenticationEntryPoint(new HttpStatusEntryPoint(HttpStatus.UNAUTHORIZED)))
            .build();
}
```

Пояснення налаштування безпеки BFF

- Ланцюг з `@Order(2)` і `securityMatcher("/bff/api/**")` стоїть між JWT-ланцюгом API та ланцюгом сторінок.
- Автентифікація береться з тієї самої сесії, яку створює форма входу, тому окремий вхід для SPA не потрібен.
- Метод `csrf.spa()` зберігає токен у куці `XSRF-TOKEN`, доступній JavaScript, і приймає його із заголовка `X-XSRF-TOKEN`.
- Після входу Spring Security очищує токен, тому клієнт перед першим POST звертається до ендпоінта `/csrf`, який записує свіжу куку.
- `HttpStatusEntryPoint` визначає відповідь неавтентифікованому клієнту: 401 замість перенаправлення на сторінку входу, тому `fetch` не отримує HTML замість JSON.

BFF-контролер камер для односторінкового клієнта

```java
@RestController
@RequestMapping("/bff/api/v1")
public class CameraBffController {

    private final CameraService cameraService;

    public CameraBffController(CameraService cameraService) {
        this.cameraService = cameraService;
    }

    @GetMapping("/csrf")
    public void csrf(CsrfToken token) {
        token.getToken(); // записує свіжу куку XSRF-TOKEN
    }

    @GetMapping("/cameras")
    public List<CameraRegistration> getCameras() {
        return cameraService.getAllCameras();
    }

    @PostMapping("/cameras")
    public ResponseEntity<CameraRegistration> addCamera(@Valid @RequestBody CreateCameraForm form) {
        var created = cameraService.registerCamera(form.cameraId(), form.location(), form.ipAddress());
        return ResponseEntity.status(HttpStatus.CREATED).body(created);
    }
}
```

Клієнтський модуль API для будь-якого фреймворку

```
// api.ts: однаково використовується з React, Angular, Vue або Svelte
const BFF = '/bff/api/v1';

export async function fetchCameras(): Promise<Camera[]> {
    const res = await fetch(`${BFF}/cameras`);
    if (res.status === 401) {
        window.location.assign('/login');
    }
    return res.json();
}

function readXsrfCookie(): string {
    const value = document.cookie
        .split('; ')
        .find(row => row.startsWith('XSRF-TOKEN='))
        ?.substring('XSRF-TOKEN='.length);
    return decodeURIComponent(value ?? '');
}
```

Надсилання змін із токеном CSRF

```
export async function createCamera(camera: NewCamera): Promise<Camera> {
    await fetch(`${BFF}/csrf`); // свіжа кука XSRF-TOKEN після входу

    const res = await fetch(`${BFF}/cameras`, {
        method: 'POST',
        headers: {
            'Content-Type': 'application/json',
            'X-XSRF-TOKEN': readXsrfCookie()
        },
        body: JSON.stringify(camera)
    });

    if (!res.ok) {
        throw new Error(`Помилка створення камери: ${res.status}`);
    }
    return res.json();
}
```

Пояснення роботи клієнта SPA з Spring BFF

- Взаємодію з сервером винесено в окремий модуль, тому компоненти будь-якого фреймворку викликають лише `fetchCameras` та `createCamera`.
- Запити йдуть на відносний шлях `/bff/api/v1`, а браузер автоматично додає сесійну куку автоматично, бо сторінка і API розташовані на одному домені.
- Перед POST клієнт оновлює куку `XSRF-TOKEN` і передає її значення в заголовку `X-XSRF-TOKEN`.
- Angular `HttpClient` та Axios за замовчуванням самі переносять куку `XSRF-TOKEN` у заголовок `X-XSRF-TOKEN`, а для `fetch` це робиться явно.
- Відповідь 401 повертає користувача на спільну сторінку входу.

Порівняння Thymeleaf та SPA у патерні BFF

- Підхід Thymeleaf зберігає монолітну цілісність, не потребує збірки JavaScript, забезпечує швидке перше відображення та має низьку складність.
- Серверний рендеринг доцільний для кабінетів адміністрування, звітів, форм та каталогів.
- SPA у патерні BFF надає динамічний інтерфейс із перемиканням екранів без перезавантаження та зберігає стан у браузері.
- Підхід SPA потребує складнішого конвеєра збірки, налаштування резервної маршрутизації та обробки CSRF.
- Гібридний підхід поєднує сторінки Thymeleaf і односторінковий клієнт в одному застосунку зі спільною сесією та окремими ланцюгами безпеки.

Типові помилки конфігурації статичних ресурсів та кешування

- Відсутність директив `Cache-Control` для важких незмінних бібліотек, шрифтів та стилів.
- Річний `Cache-Control` для файлів без хешу в імені, через що клієнти не отримують оновлень.
- Збереження користувацьких файлів (завантажень, медіа) безпосередньо в каталозі classpath.
- Стиснення статики процесором на льоту замість використання `EncodedResourceResolver`.

Типові помилки безпеки шаблонів та обробки форм

- Використання директиви `th:utext` для відображення неперевіреного користувацького тексту (XSS).
- Конкатенація параметрів запиту в імена представлень, `redirect:` або вирази SpEL (SSTI, Open Redirect).
- Покладання виключно на валідацію в браузері без перевірки на сервері через `@Valid`.
- Повернення імені представлення після успішного виконання POST замість патерну PRG.
- Використання звичайного атрибута `action` або посилань GET для зміни стану, через що запити проходять без токена CSRF.

Архітектурні антипатерни роботи з сесіями та SPA

- Збереження токенів JWT доступу в `localStorage` браузера замість кук із прапорцем `HttpOnly`.
- Довгі блокувальні виклики в контролерах, що вичерпує пул потоків веб-сервера.
- Відсутність резервної маршрутизації для SPA або повернення `index.html` зі статусом 200 на невідомі шляхи API.

Висновки

- `DispatcherServlet` розподіляє запити між контролерами, обробниками статики та представленнями Thymeleaf.
- Швидка роздача статики базується на поєднанні `Cache-Control`, `ETag`, попереднього стиснення та версіонування за хешем вмісту.
- Веб-адаптери розміщуються всередині модулів Spring Modulith і звертаються лише до публічного API модуля.
- Безпека Thymeleaf-інтерфейсу спирається на окремий сесійний ланцюг фільтрів, токени CSRF у формах, контекстне екранування та заголовки CSP.
- Патерн BFF у тому самому застосунку поєднує односторінковий клієнт будь-якого фреймворку із сесійною автентифікацією та захистом CSRF через окремий ланцюг безпеки.

Індивідуальне завдання

1. Обрати власну предметну область і створити проєкт Spring Boot 4.1 (Java 25) зі стартерами `web`, `thymeleaf`, `validation` та `security`.
2. Створити дві сторінки Thymeleaf для однієї сутності: список записів і форму додавання. Дані зберігати в пам'яті (наприклад, у `Map`), поля форми перевіряти анотаціями `@NotBlank`, `@Size` тощо, а після успішного збереження перенаправляти на список (PRG).
3. Налаштувати `SecurityFilterChain` із власною сторінкою `login.html` та користувачами `USER` і `ADMIN` в `InMemoryUserDetailsManager`, дозволивши створення записів лише `ADMIN`.
4. Показати в шаблонах ім'я користувача, кнопку виходу через POST і кнопку створення лише для `ADMIN` за допомогою діалекту `sec:`.

Індивідуальне завдання

1. Відкрити список без входу та переконатися в перенаправленні на `/login`, а після невдалого входу — у повідомленні про помилку.
2. Увійти з роллю `USER` і переконатися, що кнопку створення приховано, а надсилання форми, відкритої за прямою адресою, повертає 403.
3. Увійти з роллю `ADMIN`, надіслати некоректні дані та переконатися у відображенні помилок валідації під полями форми.
4. Створити запис зі значенням `<script>alert(1)</script>` у текстовому полі та переконатися, що текст відображається екранованим, а скрипт не виконується.
5. Знайти поле `_csrf` у коді сторінки форми та переконатися, що оновлення після створення не надсилає форму повторно.
6. Завантажити архів проєкту та звіт із даними перевірок (знімки екрана, коди відповідей).

Групове завдання

1. Додати до командного проєкту щонайменше дві сторінки Thymeleaf для ключового сценарію предметної області, наприклад перелік записів і форму створення або редагування з валідацією JSR-380 та патерном Post-Redirect-Get.
2. Винести спільні елементи сторінок (навігацію, ім'я користувача, кнопку виходу) у фрагменти `th:fragment` та підключити стилі з локальних статичних ресурсів із версіонуванням за хешем вмісту.
3. Реалізувати сторінку входу та власні сторінки помилок `error/403.html` і `error/404.html` у стилі інтерфейсу проєкту.

Групове завдання

1. Залишити захист REST API токенами JWT для шляхів `/api/**` і додати другий `SecurityFilterChain` для сторінок, де вхід виконується через форму, а захист CSRF залишається увімкненим.
2. Налаштувати вхід на сторінки для користувачів і ролей, які вже зберігаються в базі даних після попередньої лекції.
3. Показувати кнопки та розділи сторінок відповідно до ролі користувача через діалект `sec:` і перевіряти ті самі права на сервері (правилами `SecurityFilterChain` або анотаціями `@PreAuthorize`).
4. Написати звіт із деталями реалізації (ланцюги безпеки, ролі) та знімками екрана сторінок для різних ролей.